Computational Geometry: Orthogonal range searching and KD tress

Pritha Banerjee

Resources

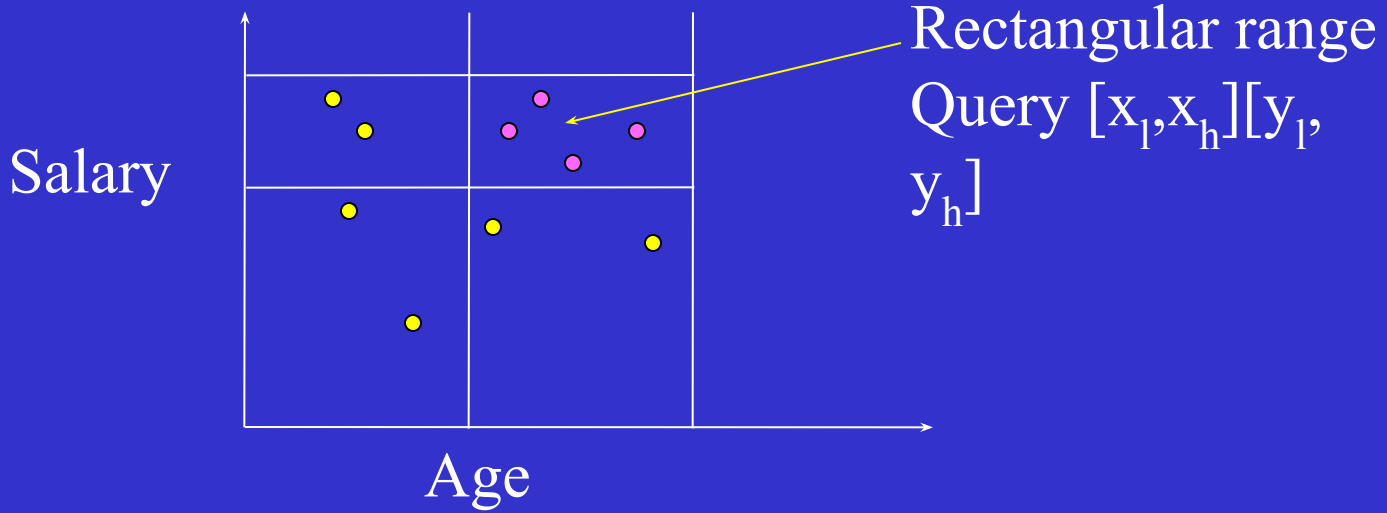
- Books and Lecture Notes:
  - Chapter 5 of Computational Geometry : Algorithms and Applications, Mark De Berg, Marc van Kreveld, Mark Overmars, Otfried Schwarzkopf, Springer
  - Lecture notes of David Mount from the web

Orthogonal Range Searching (querying a database)

Input : a query range and a set of points P

Output: Report all the points in the given range

Range query: 1 dimensional, 2 dimensional rectangular or orthogonal (axis parallel) range query, d -dimensional



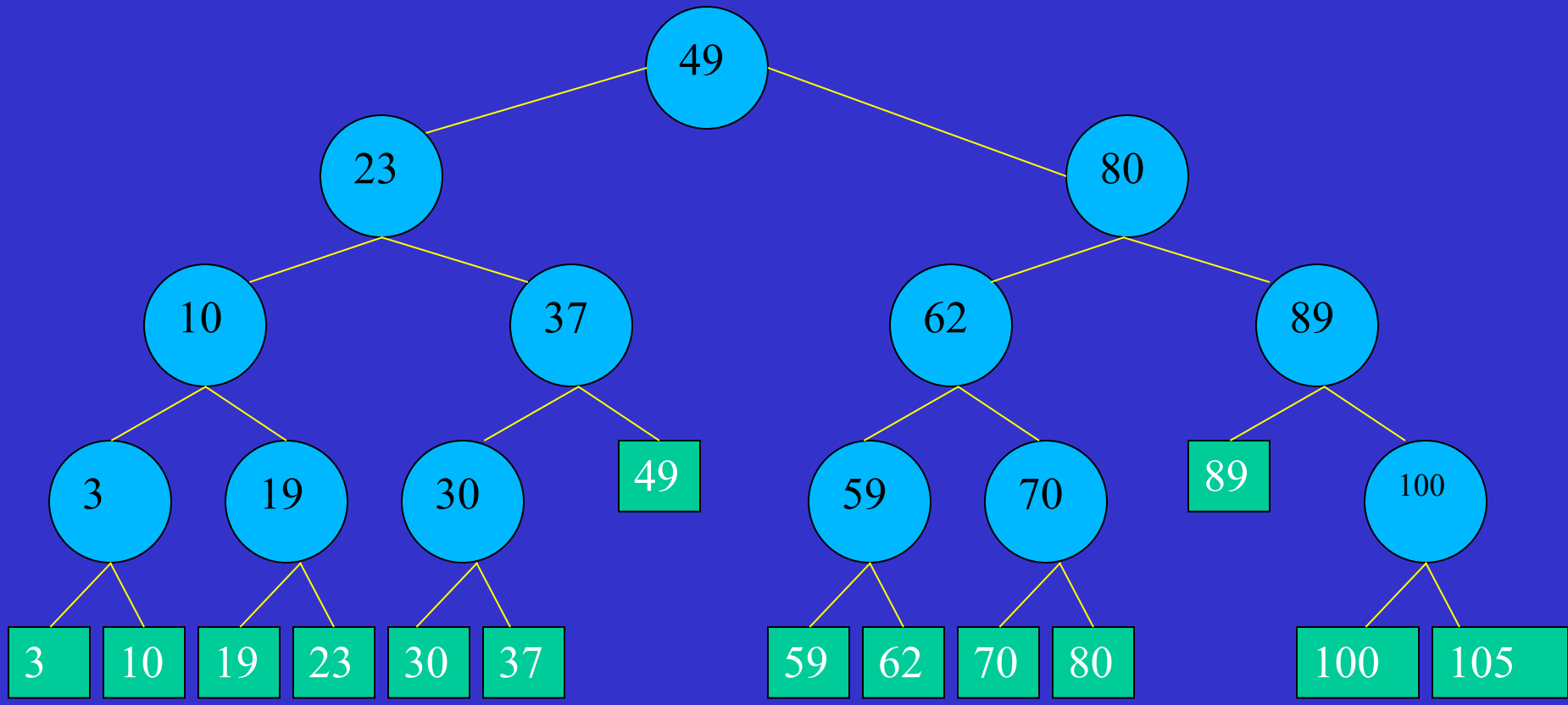
1 Dimensional Range Searching

Input: A set of real numbers $P = \{p_1, p_2, \ldots p_n\}$

Output: Report all numbers inside the 1 D query rectangle $[x_1 : x_2]$

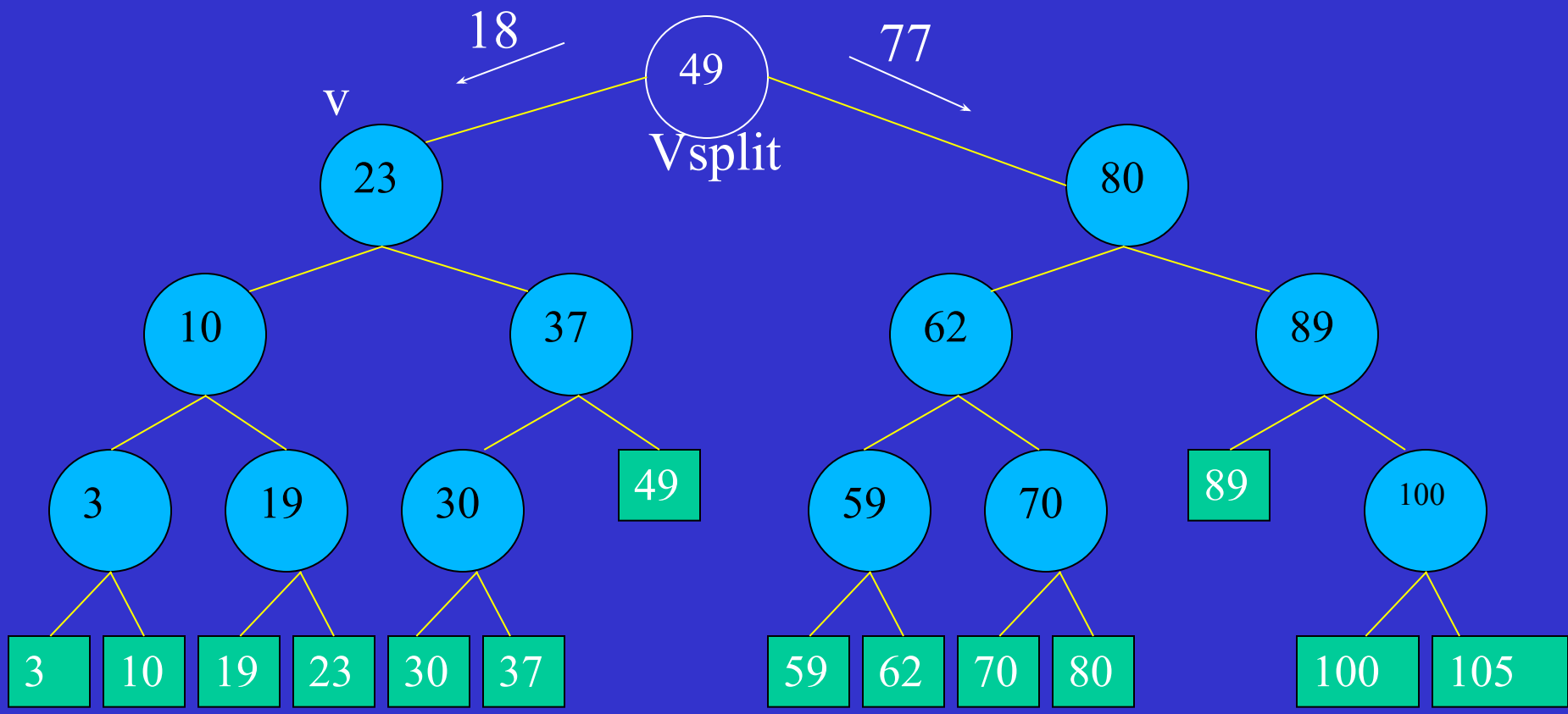
- Algorithm: Check each element if it lies in $[x_1 : x_2]$?
- Demerits :
  - What happens in Higher dimension ? 2 D range searching or more
  - What about inserting new points in the set ?
- Solution:
  - Balanced Binary Search Tree T
    - Leaves : points
    - Internal nodes: splitting values to guide search

1D range query



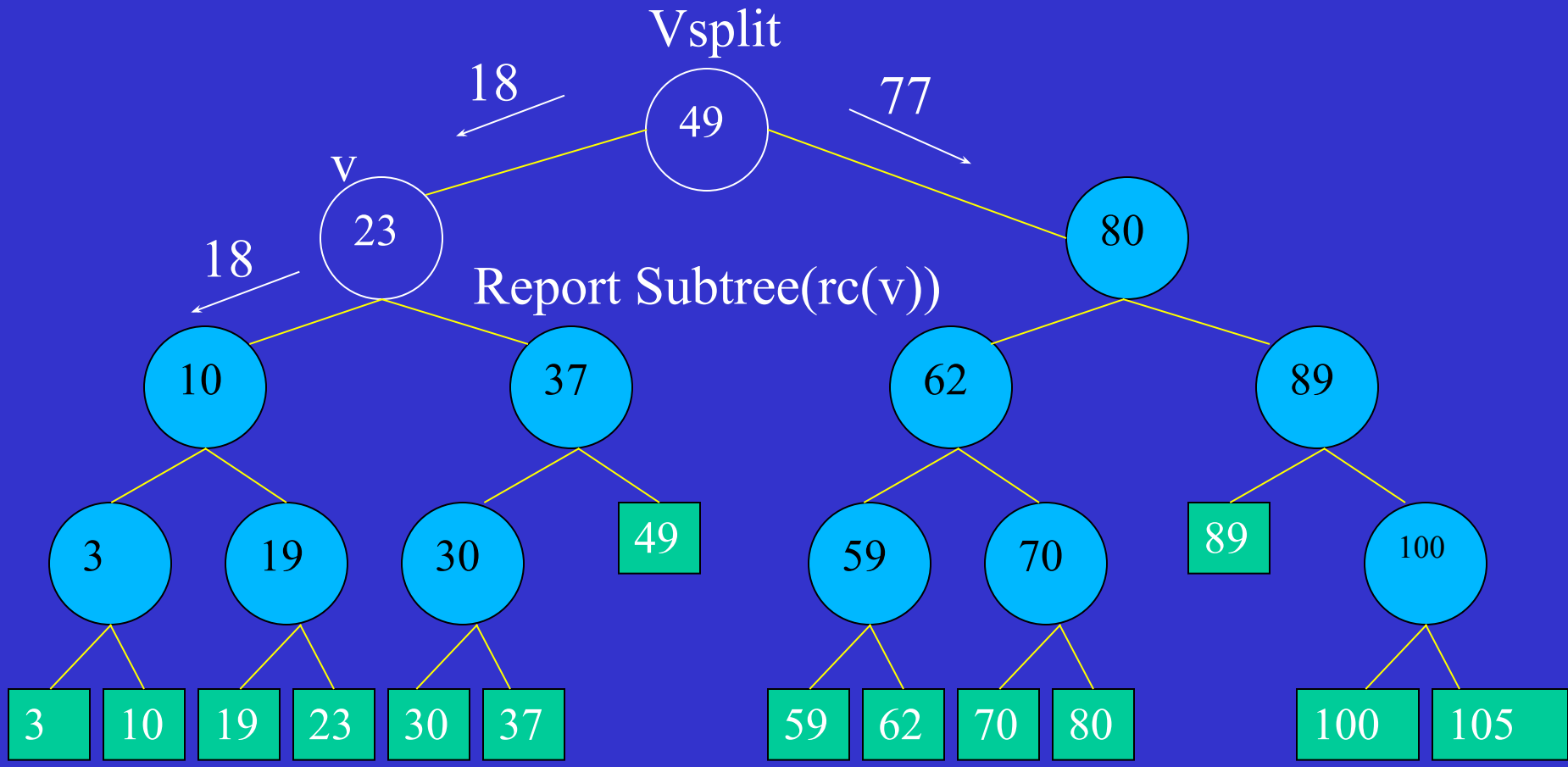
Find all points in Range [18:77]

1D range query



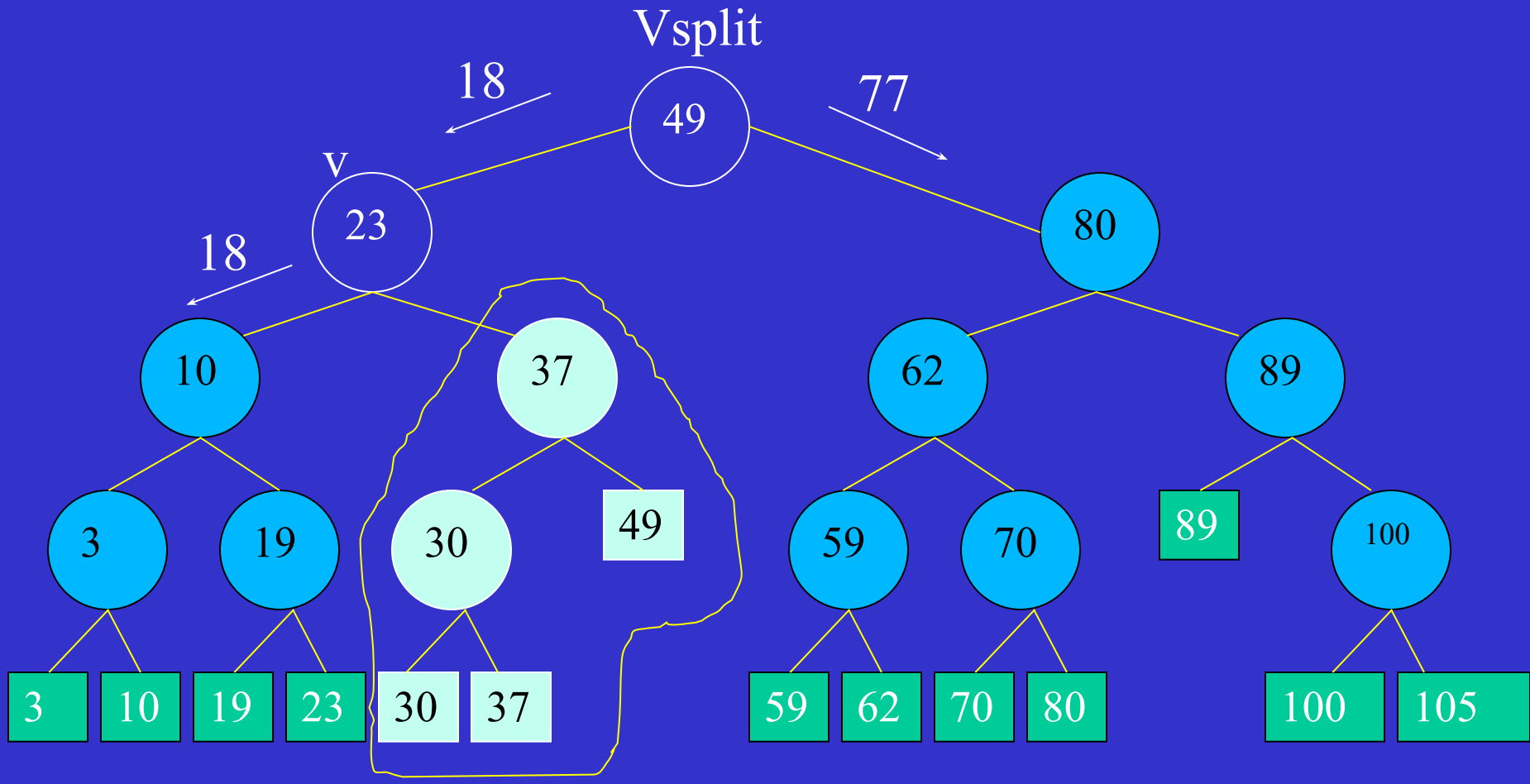
Find all points in Range [18:77]

1D range query



Find all points in Range [18:77]

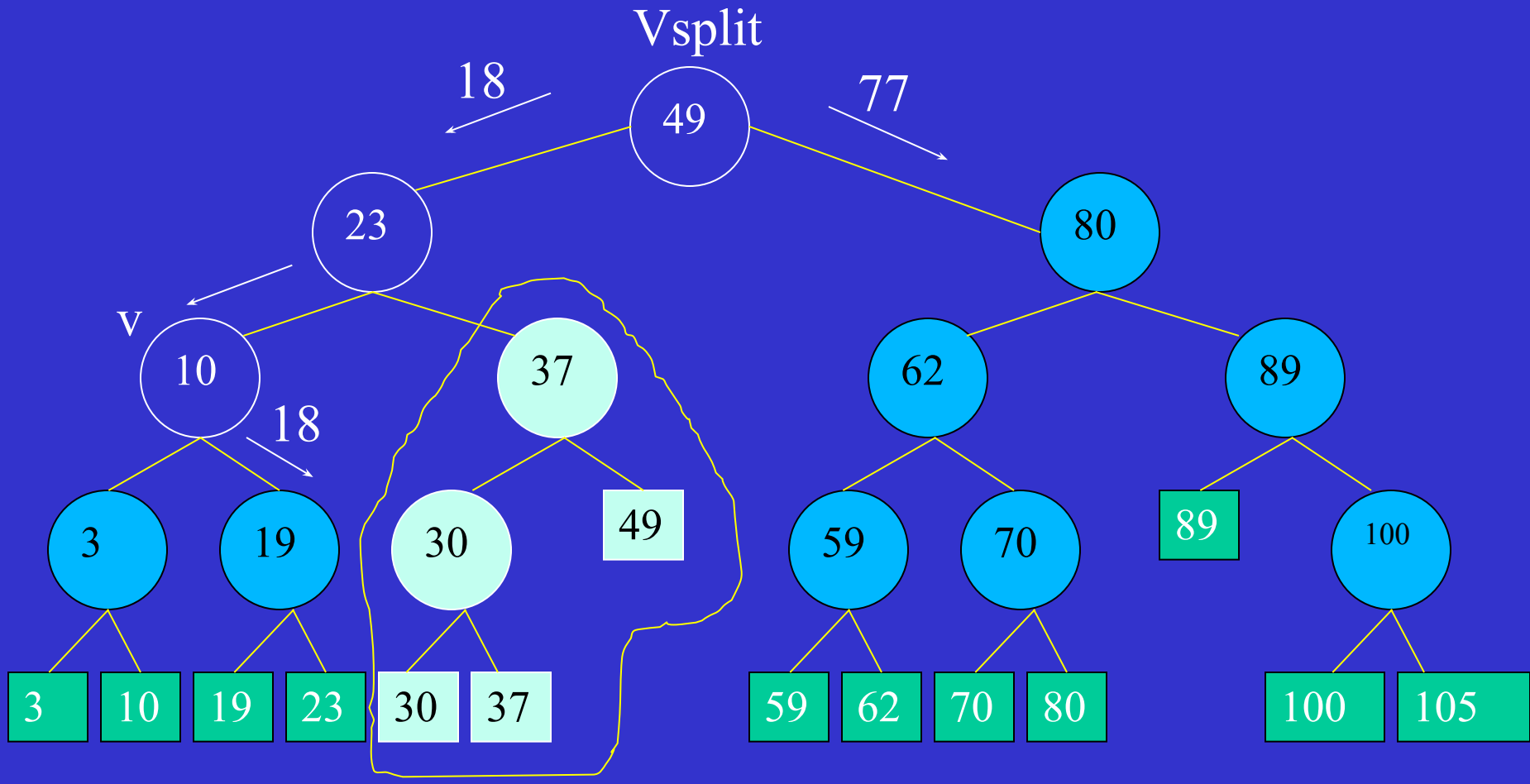
1D range query



Find all points in Range [18:77]

Reported Points: 30 37 49

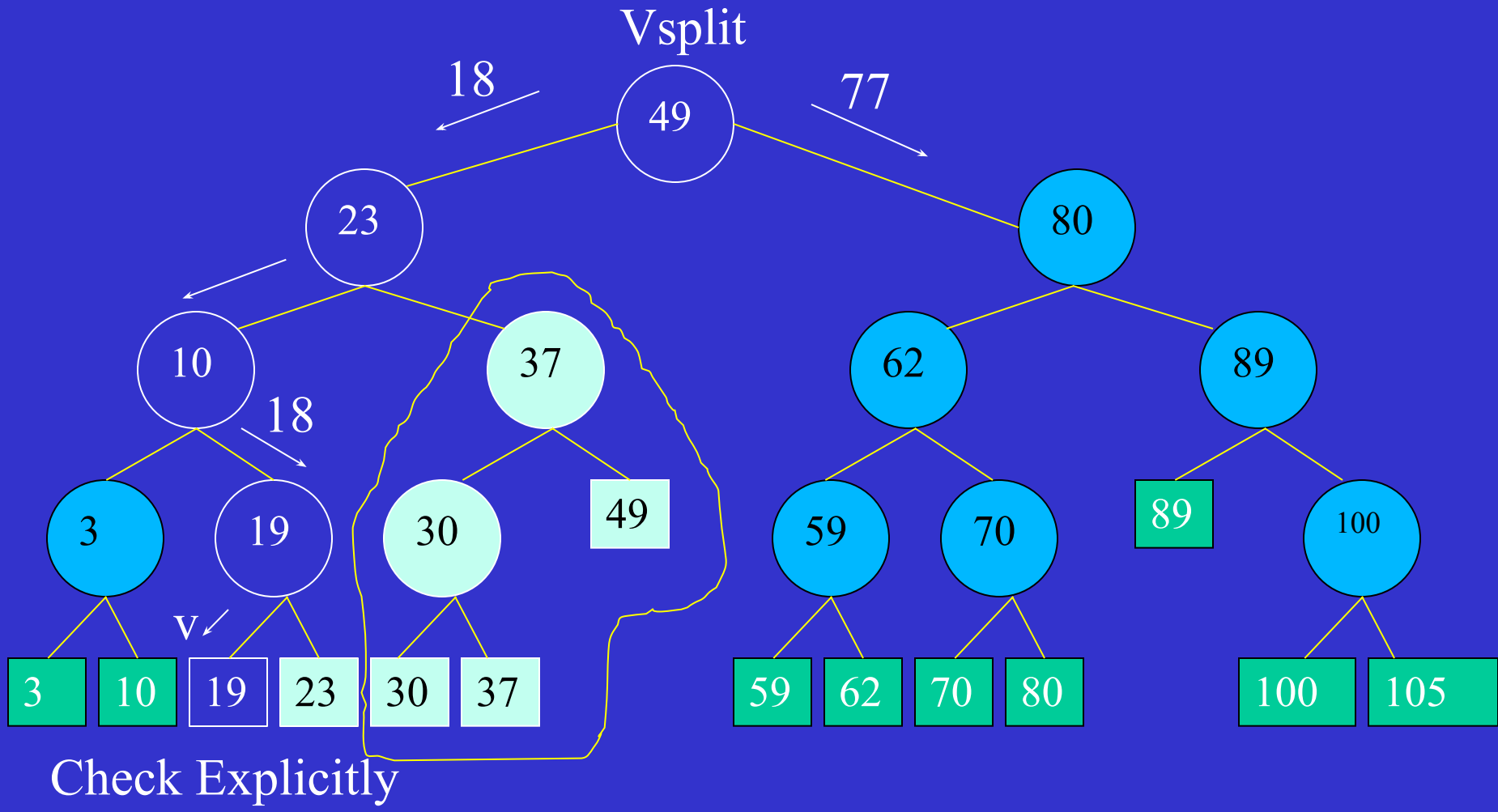
1D range query



Find all points in Range [18:77]

Reported Points: 30 37 49

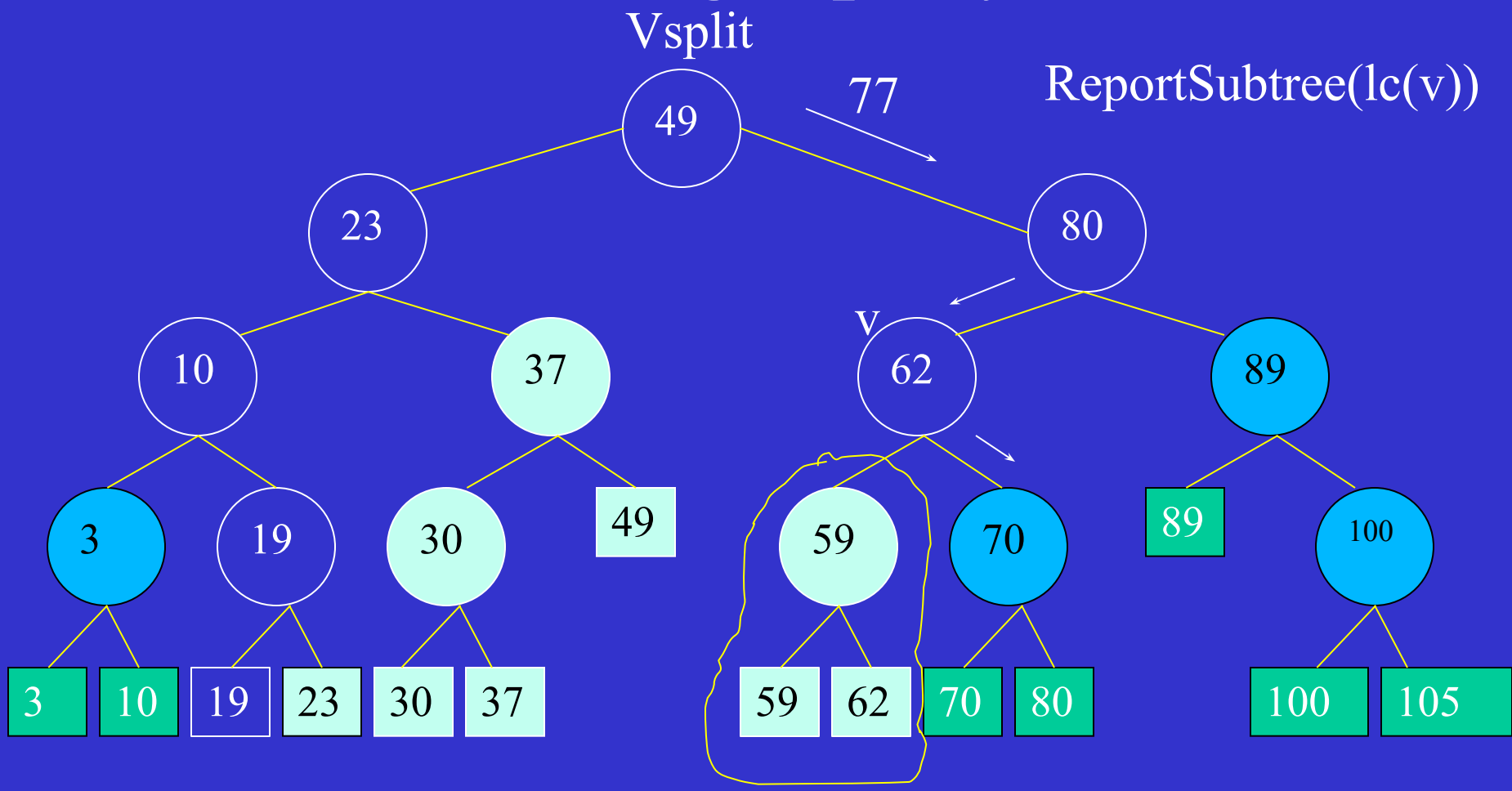
1D range query



Find all points in Range [18:77]

Reported Points: 30 37 49 23 19

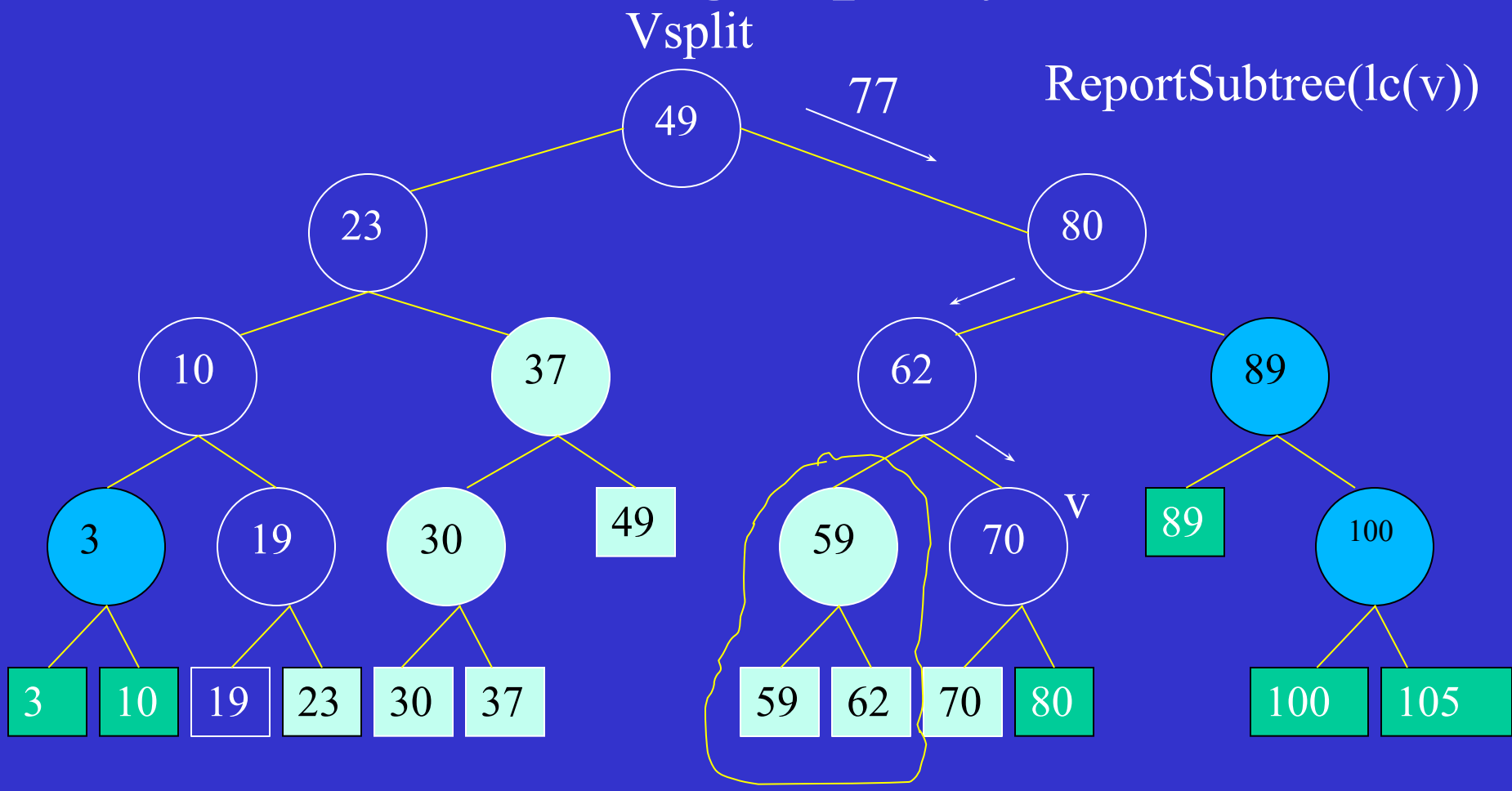
1D range query



Find all points in Range [18:77]

Reported Points: 30 37 49 23 19 59 62

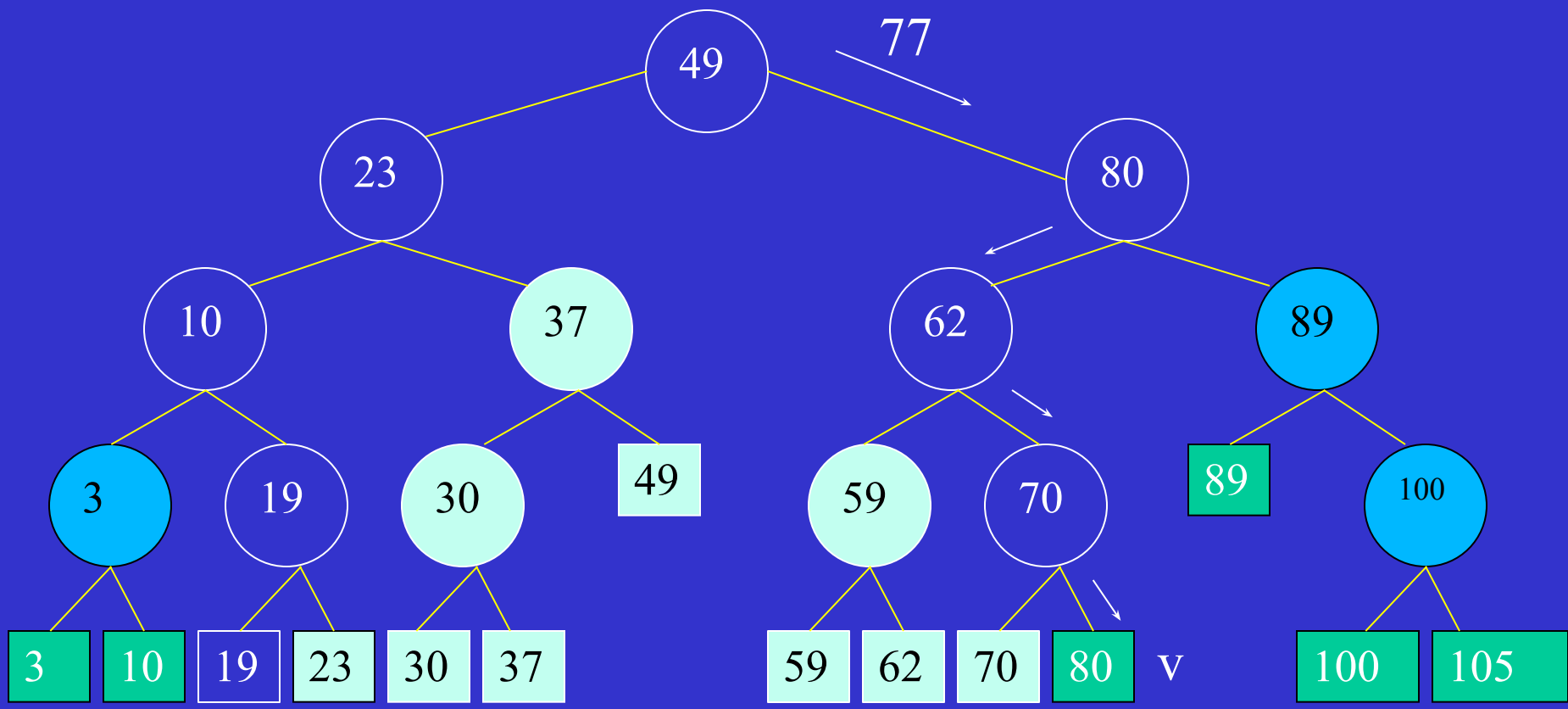
1D range query



Find all points in Range [18:77]

Reported Points: 30 37 49 23 19 59 62 70

1D range query



Check Explicitly

Find all points in Range [18:77]

Reported Points: 30 37 49 23 19 59 62 70

Algorithm

- FindSplitNode(T, x_1, x_2) : Find a node where both children of the node need to be searched
- ReportSubtree(v) : Traverse subtree rooted at v and report the points stored in leaves
- 1D Rangequery(T, x_1, x_2) uses above two procedures.

Algorithm : FindSplitNode

- FindSplitNode(T, x_1, x_2)
- Output: return v where path to $x_1$and x_2 splits

v = root(T)

while v is not leaf and ($x_1 > x_v$ or $x_2 <= x_v$)

    if $x_2 <= x_v$ then v = lc(v)

    else v = rc(v)

return v

$x_2 <= x_v$: [x_1 , x_2] is to the left of node v

$x_1 > x_v$: [x_1 , x_2] is to the right of node v

Algorithm: 1DRangeQuery

- Input: Balanced BST and $[x_1 , x_2]$
- Output: All points in $[x_1 , x_2]$

Vsplit = FindSplitNode(T, x_1 , x_2)

if Vsplit is leaf check if this point has to be reported

else

v = lc(Vsplit)

while v is not leaf

if $x_1 < x_v$ then ReportSubTree(rc(v))

v = lc(v)

else v = rc(v)

Check if v is to be reported

Do the same for x_2

Space and Time Complexity

- Space: O(n) since BST
- Time : Construction of BST + Query time
- BST Construction : O(n log n)
- Query time : O(log n + k) , k : # of points reported (Output sensitive)
- ReportSubTree(): traverses all nodes, linear
  - Nodes visited by all calls of ReportSubTree() : O(k)
- How many other nodes visited? Nodes in search paths of x1,x2; paths have length O(log n)
- Total query time complexity: O(log n + k)

Does Algo reports all points ?

- Lemma: Algo reports exactly those points that lie in the query range

Show that, any reported point p is in range $[x_1 : x_2]$

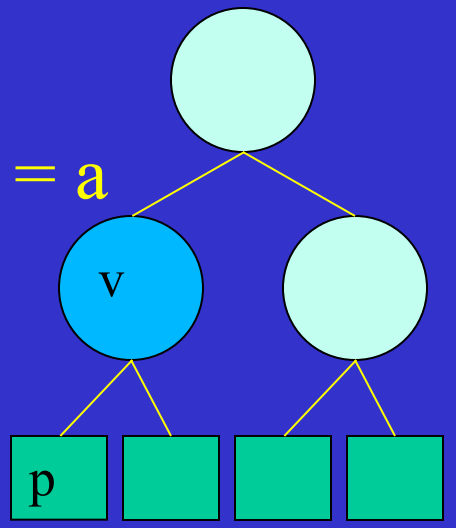
- If path ends at leaf where p stored, test explicitly
- Else points are reported in ReportSubTree()
- v : node on path when p is reported by RST(rc(v))
  - v, rc(v) in left subtree of vsplit; $p <= x_{vsplit}$
- Since search path of x_2 goes right of vsplit; $p < x_2$
- Search path of x_1 goes left of v & p is in right ST(v), $x_1 < p$; p lies in interval $[x_1 : x_2]$
- Similarly prove p lies in range when it follows the path to x_2

Does Algo reports all points ?

Show that any point p in range is reported

a: leaf node where p is stored

v: lowest ancestor of a visited by algo; claim v = a implies p is reported



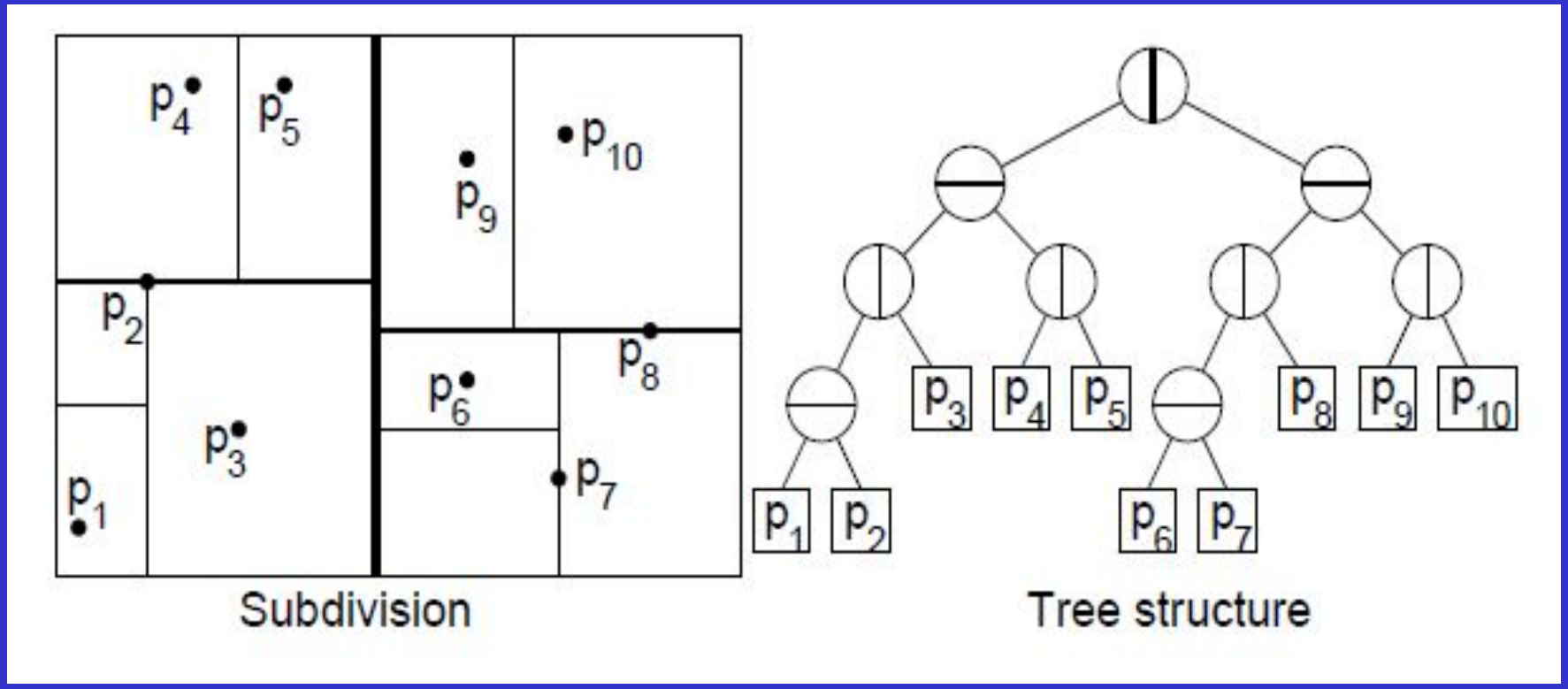
Proof by contradiction: Assume : $v \neq a$

- v can not be a node reported by RST
- v can only be on the search path of x_1 or x_2 or both
- if a is on left ST of v => search path of x_1 goes to the right (else v will not be lowest ancestor) => $p < x_1$: contradiction
- Similarly, if a is on right ST of v => search path of x_2 goes left => $p >= x_2$; contradiction

Kd Trees

- Input : set of points P, p := (px,py), query rectangle $[x_1:x_2]$ x$[y_1:y_2]$
- Two one dimensional sub query, one on x coordinate, other on y coordinate
- How to arrange points in a BST?
  - odd depth : x coordinate; recursively divide the points on x coordinate
  - Even depth : y coordinate; recursively divide the points on y coordinate
  - Store the splitting value at node
  - The subtree corresponds to a region on the plane

KD tree and the points in plane



Build KD-tree

- I/p: set of points P , current depth ,O/p: root of KD tree storing P

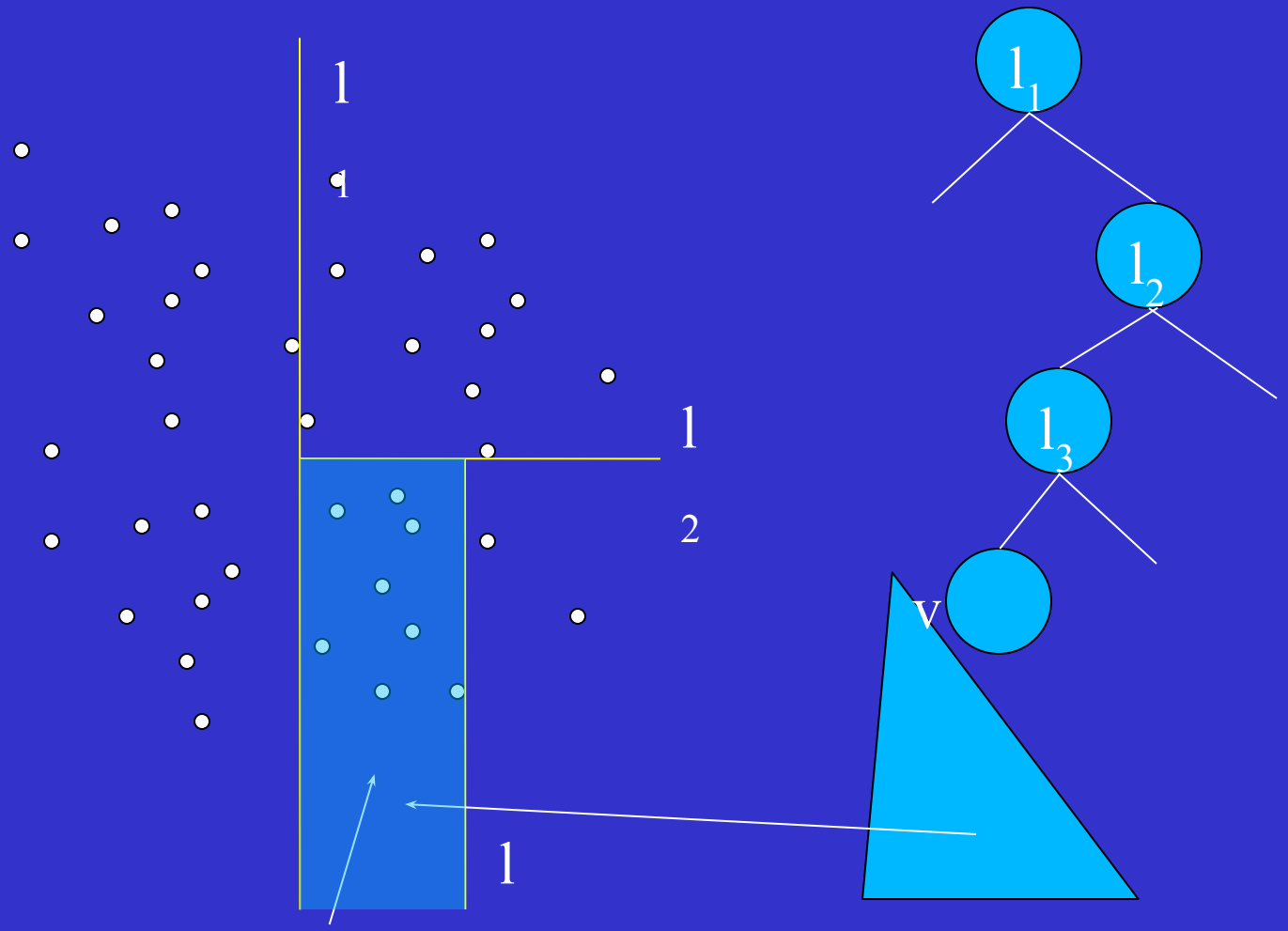
- if P has single point return leaf storing this point
- else
  - if depth is even then split P to P1 and P2, with vertical line l thr' median x-coordinate of points in P. P1 : left of l and P2 : right of l
  - Else if depth is odd then split P to P1 and P2, with horizontal line l thr' median y-coordinate of points in P. P1 : below l and P2 : above l
- vleft = BuildKDTree(P1,depth+1)
- vright = BuildKDTree(P2,depth+1)
- create node v storing l, vleft and vright are left and right child of this node
- return v

Build KD-tree: Complexity

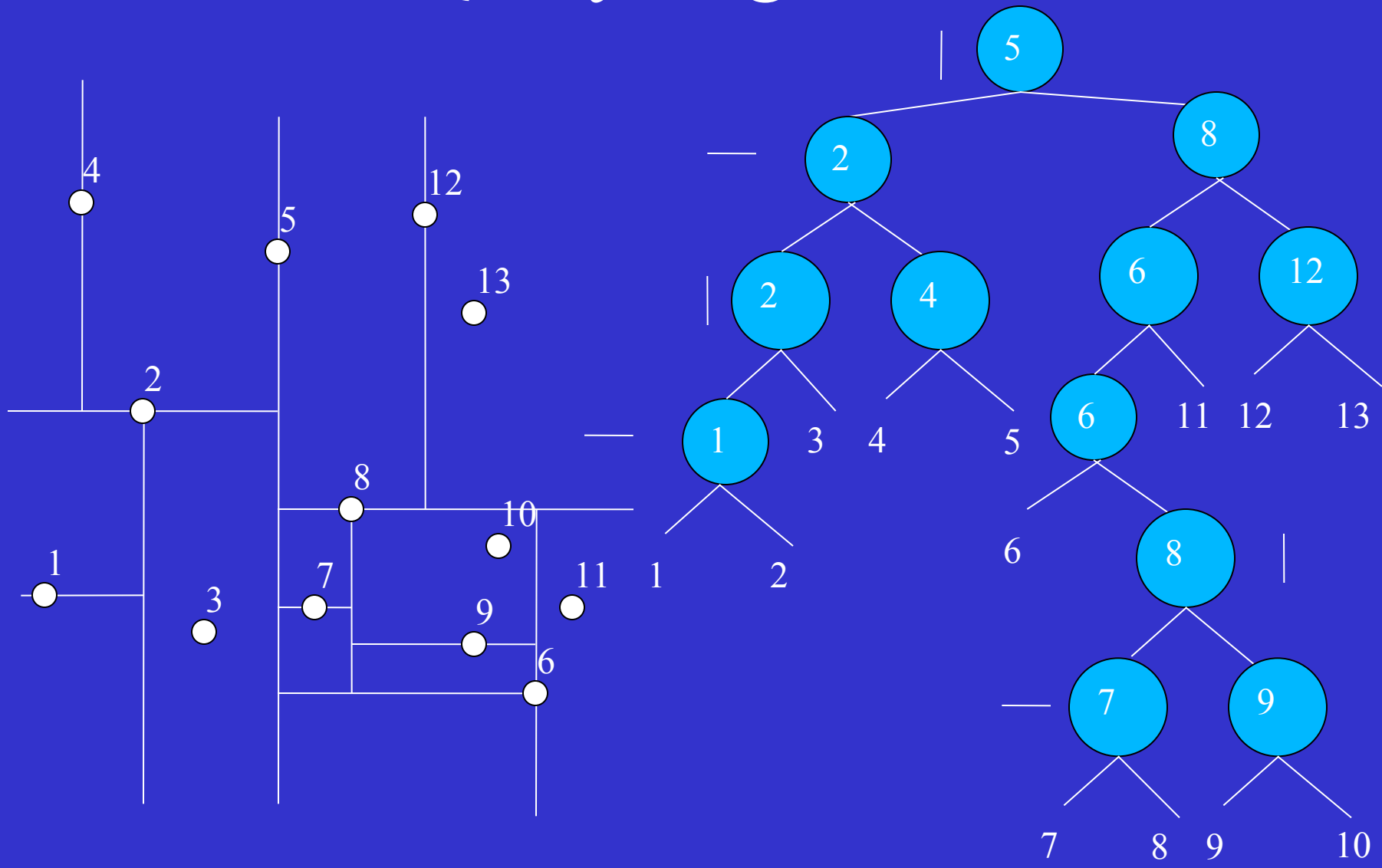
- Construction time: O(n log n), Space: O(n)
- Find the split time: median finding
  - Pre-sort the points on x coordinate
  - Pre-sort the points on y coordinate
- Divide and conquer
  - Split sorted list at median s and put p < s in left and p> s in right node; O(n)
  - Recursively for left and right child
- $T(n) = O(1)$; $n = 1$

  $= O(n) + 2T(\text{ceil}(n/2))$, $n > 1$
- O(n log n)

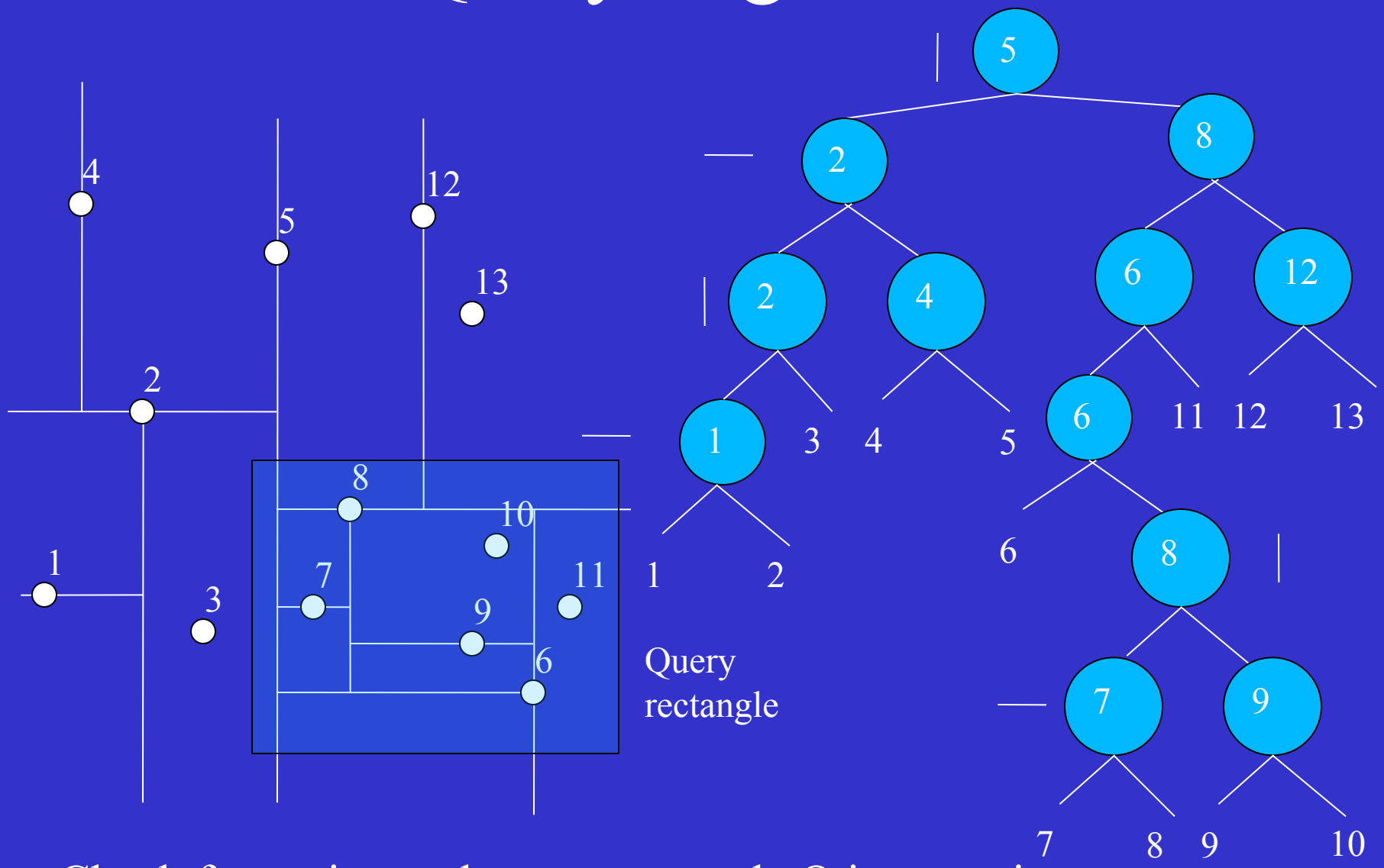
Query Algorithm: Region



Query Algorithm



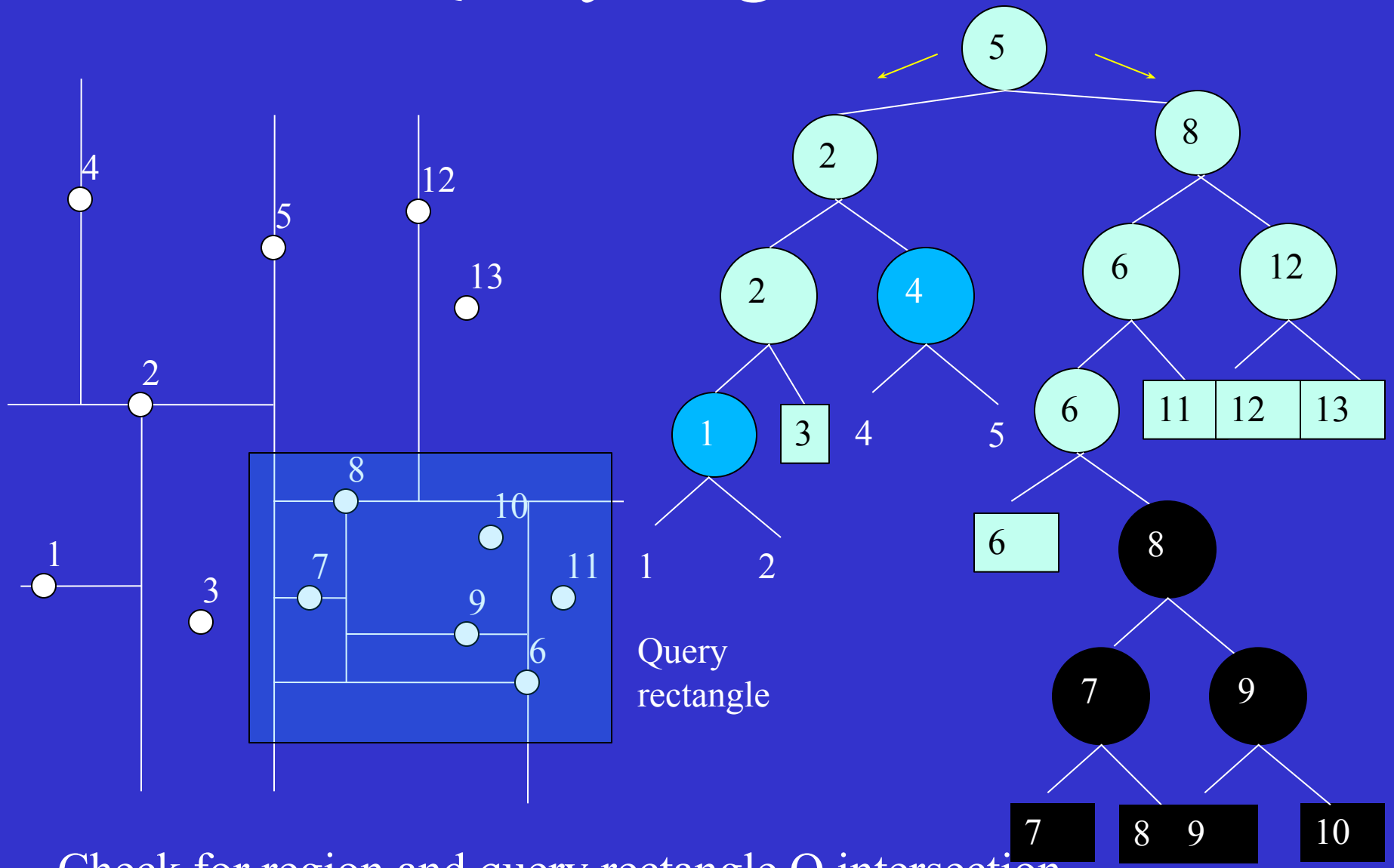
Query Algorithm



Check for region and query rectangle Q intersection

Region < Q : report all points in Subtree corresponding to Region

Query Algorithm



Check for region and query rectangle Q intersection

Region < Q : report all points in Subtree corresponding to Region

QueryAlgo: SearchKdTree(v, Q)

- I/p: root of a kd tree (subtree), Q: range
- O/p: all points at leaves below v that lies in range
- If v is leaf report point at v if lies in Q and return
- If region(lc(v) is fully in Q, ReportSubTree(lc(v))

  Else if region(lc(v)) intersects with Q,

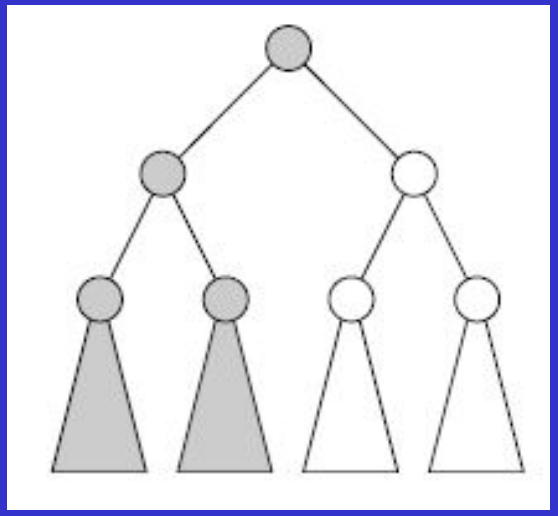
  SearchKdTree(lc(v), Q)
- If region(rc(v) is fully in Q, ReportSubTree(rc(v))

  Else if region(rc(v)) intersects with Q,

  SearchKdTree(rc(v), Q)

Region R is fully in Q or intersecting? the internal node v defines a rectangular region on the x-y plane . Let R be [xl, xr] and [yb, yt] If x1 <=xl <= xr <= x2 and y1 <= yb <= yt <= y2 then R fully in Q.

QueryAlgo: Complexity



- Always go down two steps in tree
- At level 2: 2 search paths out of n/4 paths

$Q(n) = 2 + 2Q(n/4);$

- time to traverse a subtree and report the points stored in its leaves is linear in the number of reported points k, ie, O(k), where k is the total number of reported points.
- bound the number of nodes visited by the query algorithm that are not in one of the traversed subtrees
- For each such node ν, the boundary of the query range intersects region(ν).
- Consider the number of regions intersected by any vertical line (number of regions intersected by left and right edges of Q).
- Similar upper bound for top and bottom edges of Q considering intersection with horizontal line

QueryAlgo: Complexity

- Consider vertical edge of Q, edge is either to left or right of the vertical line l; if v is at depth 1, edge of Q intersects only one region
- If v is at depth 2, Q edge may intersect 2 regions; here out of 4, Q edge may intersect atmost 2 regions, # of nodes at depth 2 is (n/4).
- Q(n) : # of intersected regions in kd tree storing n points whose root contains a vertical split line.
- Always go down two steps in tree
- Q(n) = 2 + 2Q(n/4); where the first 2 is due to l intersecting region of root and one of its children
- $O(\sqrt{n} + k)$

Building generalized Kd-tree

- At the root, split the set of points into two subsets of roughly the same size based on median of first coordinate of the points.
- At the children of the root the partition is based on the second coordinate, third coordinate, and so on, until at depth $d - 1$
- At depth d we start all over again, partitioning on first coordinate.
- The recursion stops when there is only one point left, which is stored at a leaf.
- Because a d-dimensional kd-tree for a set of n points is a binary tree with n leaves, it uses O(n) storage.
- The construction time is O(n log n).

Kd-tree Query of dimension d

- Nodes in a d-dimensional kd-tree correspond to regions, as in plane.
- The query algorithm visits those nodes whose regions are properly intersected by the query range, and traverses subtrees to report the points stored in the leaves that are rooted at nodes whose region is fully contained in the query range.
- Query time is bounded by $O(n^{(1-1/d)} + k)$.

Limitations:

- when the number of reported points is small, the query time is relatively high.

Note:

- kd-tree is a special case of a more general class of hierarchical spatial subdivisions, called binary space partition trees (or BSP trees) in which splitting lines (or hyperplanes in general) may be oriented in any direction.